

CSE 220: Systems Fundamentals I

Stony Brook University Homework Assignment #8 SPR 2026

Due: Friday, 4/8/2025 by 11:59 pm Eastern Time (Hard deadline, no extension in any case.)

You will submit your zipped code to BrightSpace. Zipped file should be named <SBUID#>-LastName-FirsttName-HW8.zip

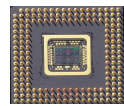
Learning Outcomes

After completion of this programming project, you should be able to:

- Have a good understanding of MIPS assembly programming.
- Implement non-trivial algorithms that require conditional execution and iteration.
- Read and write one/two-dimensional character arrays of arbitrary length.
- Design and implement functions that conform to the MIPS assembly function-calling conventions.
- This assignment will be marked out of 100.

Overview

Your task in this assignment is to implement various functions as specified below. If you are having difficulty implementing these functions, consider writing out pseudocode or implementing the algorithms in a higher-level language first. Once you understand the algorithm and what steps to perform, translate the logic to MIPS assembly code.



Preliminaries

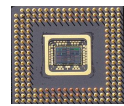
- When writing assembly code, try to stay consistent with your formatting and to comment as much as possible.
- You must implement the programming assignments in the MIPS Assembly language by yourself. You may not write or use a code generator or other tools that write any MIPS code for you. You must manually write all MIPS assembly code you submit as part of the assignments.
- Do not copy or share code.
- Submit your final *.asm files to Brightspace. Code that crashes or cannot be run will earn no credit. No changes to your submission will be permitted once the deadline has passed.

How Your MIPS Code Will Be Graded

As with the C assignments, grading scripts will execute your code with input values (e.g., command-line arguments, function arguments) and will check for expected results (e.g., print-outs, return values, etc.). For this assignment, TAs will run your program manually. Therefore, you must generate input and output exactly as asked. You must implement the “print statements” for input and output exactly as specified and requested in the assignment.

Some other items you should be aware of:

- Each test case must execute in 10000 instructions or fewer (you should be able to complete each part with 300 instructions- my good guess).
- Any excess output from your program (e.g., debugging notes) will affect grading. **Do not leave erroneous print-outs in your code.**
- We will provide you with a small set of test cases for each assignment to give you a sense of how your work will be graded. It is your responsibility to test your code thoroughly by creating your test cases. Provide these tests in the Readme file as well.
- You need to give your ID and name in the Readme file as well, and in each assembly file.
- YOUR CODE SHOULD BE THOROUGHLY COMMENTED. 50% marks



will be deducted for poor comments.

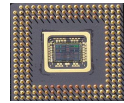
Register Conventions

You must follow the register conventions taught in the lecture and reviewed in the recitation. Failure to follow them will result in 70% credit loss when we grade your work. Below is a summary of the register conventions and how your use of them will impact grading. See the lecture notes for more details.

- The callee's responsible for saving any \$s registers it overwrites by saving copies of those registers on the stack and restoring them before returning.
- If a function calls a secondary function, the caller must save \$ra before calling the callee. In addition, if the caller wants a particular \$a, \$t, or \$v register's value to be preserved across the secondary function call, the best practice would be to place a copy of that register in an \$s register before making the function call.
- A function that allocates stack space by adjusting \$sp must restore \$sp to its original value before returning.
- Registers \$fp and \$gp are treated as preserved registers for this course. If a function modifies one or both, it must restore them before returning to the caller. There is no reason for your code to touch the \$gp register, so leave it alone.
- Please provide extensive comments. At least one comment for each instruction. Poor commenting will result in a loss of credit.

The following practices will result in a loss of 70% credit:

- "Brute-force" saving of all \$s registers in a function or otherwise saving \$s registers that are not overwritten by a function.
- Callee-saving of \$a, \$t, or \$v registers as a means of "helping" the caller.
- "Hiding" values in the \$k, \$f, and \$at registers or storing values in main memory by way of offsets to \$gp. *This is basically cheating or, at best, a form of laziness, so don't do it. We will comment out any such code we find. You have been warned as well not to use these registers in class as well.*



Part A (hw8a.asm) (Drawing with MIPS):

Write a program that asks if the user wants a triangle, a square, or a pyramid. It then asks the user for the size of the object (the number of lines it takes to draw the object). The program then writes/draws a triangle (right-angled), a square, or a pyramid of stars "*" to the console.

Some Examples/Test Cases:

(Square of height 6)

```
*****
*****
*****
*****
*****
*****
```

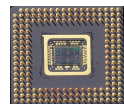
(Triangle of height 6)

```
*
**
***
****
*****
*****
```

(Pyramid of height 4)

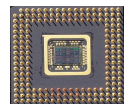
```
  *
 * *
* * *
* * * *
```

Write a subroutine for each figure. In them, use a subroutine `print_star_line` that writes a line of a given number of stars. (that number is passed as an argument to the `print_star_line` function).

**Sample output for the problem using MARS 4.5:**

```
Triangle(0) or Square(1) or Pyramid (2)?0
Required size? 3
*
**
***

-- program is finished running --
```



Part B (hw8b.asm): Swapping data between arrays

In this program, the content of two one-dimensional **integer** arrays will be exchanged (swapped). The swapping operation will be performed for each array location. The swapping operation for two arrays A and B will be performed as follows:

```
Temp = A[i]
A[i] = B[i]
B[i] = temp
```

You will implement the swapping operation in MIPS. Both arrays will be static and defined in the data stack (.data) as:

```
.align 2
```

```
A: .space      40 # memory space for Array A TAs will change this
```

```
B: .space      40 # memory space for Array B
```

```
num: .word      10 # number of elements (integer)
```

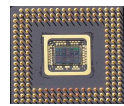
TAs can change these values. However, the size of arrays (.space) will be $\geq \text{num} \times 4$

Hint:

```
li $t0, 40    # total bytes, TA will change this
```

```
srl $t1, $t0, 2 # divide by 4 → number of elements
```

This example is for 10 integer elements. You need to take care of the elements' size and clearly define it. You need to prompt the user for input for each element of arrays A and B, then print the swapped values at the end. Please see the following snapshot of the MARS terminal. The left-hand side is the input values from the user, and the right-hand side is the swapped values.



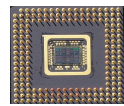
```
A[1]=1
B[1]=2
A[2]=3
B[2]=4
A[3]=5
B[3]=6
A[4]=7
B[4]=8
A[5]=9
B[5]=10
A[6]=11
B[6]=12
A[7]=13
B[7]=14
A[8]=15
B[8]=16
A[9]=17
B[9]=18
A[10]=19
B[10]=20
```

2 1|4 3|6 5|8 7|10 9|12 11|14 13|16
15|18 17|20 19|
-- program is finished running --

Figure 2.1: Output with 10 elements for Arrays A and B using MARS 4.5

Here is what is expected from your code (see the above Figure). You will ask the user for several elements. Then, you will ask the user for each element of the array in the way it is printed in the figure; input for A[1], B[1], ... is printed on the screen, and values are being asked from the user. Here, we are using 10 elements. Therefore, we have a total of 20 inputs: 10 for array A and 10 for array B. **In the end, you will print the swapped result in the format shown in Figure 2.1. 2 1 are the first elements of A and B respectively that are swapped (See the left-hand side as well. A[1]=1 and B[1]=2. After swapping, A[1]=2 and B[1]=1). We have “|” between the swapped elements. The second elements for the arrays after swapping are 4, 3, and so on.**

Any other output format will not be accepted and will result in a mark deduction.



Part C (hw8c.asm): Sorting in MIPS Assembly Language

In this problem, you will explore how to sort an integer array in MIPS assembly language using the bubble sort algorithm. Use the following C code for your implementation.

Sorting Code in C Language

```
#include <stdio.h>

// Bubble sort function (descending order)
void bubbleSort(int arr[], int n) {
    int i, j;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (arr[j] < arr[j + 1]) {
                // swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

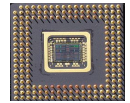
int main() {
    int v[] = {8, 100, 0, 3, 7, 9, 2, 7, -3, 0};
    int n = 10;

    printf("Sorted Array: ");

    // function call (matches your intent)
    bubbleSort(numbers, n);

    // print array
    for (int i = 0; i < n; i++) {
        printf("%d ", numbers[i]);
    }

    return 0;
}
```

```
.data

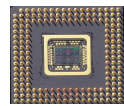
v: .word 8, 100, 0, 3, 7, 9, 2, 7, -3, 0 #create an array which holds numbers
n: word 10                               # number of elements

message: .asciiz "Sorted Array: "        #message to be printed

.text

main:

bubble_sort:
```



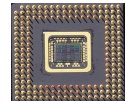
Part D (hw8d.asm): Matrix Multiplication in MIPS Assembly Language

Write a MIPS program that performs the multiplication of two matrices **A** and **B**, and stores the result in **C**.

Each matrix is a 2D array of integers (you may assume all matrices are 4×4 for simplicity).

Your program should run and print the C matrix. Assume A and B have compatibility for matrix-matrix multiplication.

```
void multiply(int n, int m, int p, int A[n][m], int B[m][p], int C[n][p])
{
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < p; c++) {
            int sum = 0;
            for (int i = 0; i < m; i++) {
                sum += A[r][i] * B[i][c];
            }
            C[r][c] = sum;
        }
    }
}
```



For the following matrices:

```
=====
# Program: 4x4 Matrix Multiplication
#=====
# C = A x B
# Each matrix is 4x4 (integers)
#=====

.data

#-----

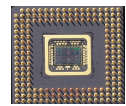
# Matrices
#-----

A: .word 1, 2, 3, 4
   .word 5, 6, 7, 8
   .word 9, 10, 11, 12
   .word 13, 14, 15, 16

B: .word 1, 0, 0, 0 # This is a unit matrix
   .word 0, 1, 0, 0
   .word 0, 0, 1, 0
   .word 0, 0, 0, 1

.align 2

C: .space 64      # 16 integers * 4 bytes each = 64 bytes
n: .word 4        # matrix dimension (4x4)
newline: .asciiz "\n"
space:  .asciiz " "
```



The output should be:

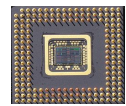
```
program is finished running

1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

-- program is finished running --
```

Clear

The TAs will change the dimensions and data to test your implementation. They can change it to 8 x 8 and fill the elements accordingly. Your program should work if done correctly.



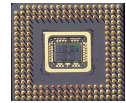
Testing & Grading Notes

- The assignment will be marked by having the TAs run each part manually using MARS 4.5.
- Add at least 5 test cases for each part. The TAs will have their evaluation.
- Comments are important. We will read and deduct reasonable marks if they are not clear.
- Write your SBU ID and Name in the README file and in each *.asm file
- Remember, if the program passes your test cases, there is a high chance the code works and is correct.

Academic Honesty Policy

Academic honesty is taken very seriously in this course. By submitting your work for grading, you indicate your understanding of, and agreement with, the following Academic Honesty Statement:

1. I understand that representing another person's work as my own is academically dishonest.
2. I understand that copying, even with modifications, a solution from another source (such as the web or another person) as a part of my answer constitutes plagiarism.
3. I understand that sharing parts of my homework solutions (text write-up, schematics, code, electronic or hard copy) is academic dishonesty and helps others plagiarize my work.
4. I understand that protecting my work from possible plagiarism is my responsibility. I understand the importance of saving my work so that it is visible only to me.
5. I understand that passing information that is relevant to a homework/exam to others in the course (either lecture or even in the future!) for their private use constitutes academic dishonesty. I will



only discuss material that I am willing to openly post on the discussion board.

6. I understand that academic dishonesty is treated very seriously in this course. I understand that the instructor will report any incident of academic dishonesty to the University's Academic Judiciary.

7. I understand that the penalty for academic dishonesty might not be immediately administered. For instance, cheating on a homework assignment may be discovered and penalized after the grades for that homework have been recorded.

8. I understand that buying or paying another entity for any code, partial or in its entirety, and submitting it as my work is considered academic dishonesty.

9. I understand that there are no extenuating circumstances for academic dishonesty.